

Шпаргалка по Ајах

Асинхронные запросы из браузера: fetch и async/await, отправка данных и файлов, отмена и таймауты, обработка ошибок, CORS и куки, устаревший XMLHttpRequest, а также приёмы для живых интерфейсов.

Что это такое

Ајах — это обмен данными с сервером без перезагрузки страницы. Исторически название расшифровывалось как «асинхронный JavaScript и XML», но сегодня обмен почти всегда идёт в JSON, а вместо XMLHttpRequest применяют fetch.

Простой запрос

```
const res = await fetch('/api/users');
if (!res.ok) {
  throw new Error(`HTTP ${res.status}`);
}
const users = await res.json();
```

Важно: fetch не считает ошибкой ответы 404 и 500 — промис отклоняется только при сетевом сбое. Всегда проверяйте `res.ok`.

Отправка данных

```
const res = await fetch('/api/users', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ name: 'Аня' }),
});

// Форма и файлы: заголовок не ставим,
// браузер подставит его сам вместе с границей
const form = new FormData(formElement);
form.append('avatar', fileInput.files[0]);
await fetch('/api/upload', { method: 'POST', body: form
});
```

URLSearchParams

Данные формы в классическом виде `a=1&b=2`

`new URL('/api/users', location.origin)`

Сборка адреса; `url.searchParams.set('page', 2)` добавит параметры

Чтение ответа

`res.json()`

Разбор JSON; бросает исключение, если тело пустое или это HTML

`res.text()` / `res.blob()`

Текст; двоичные данные для файлов и картинок

`res.status`, `res.ok`, `res.redirected`

Код ответа, признак успеха, был ли переход

`res.headers.get('content-type')`

Заголовки ответа

`res.body`

Поток: позволяет читать длинный ответ по частям

Ошибки и повторы

```
async function request(url, options = {}, retries = 2) {
  try {
    const res = await fetch(url, options);
    if (res.status >= 500 && retries > 0) {
      await new Promise(r => setTimeout(r, 500));
      return request(url, options, retries - 1);
    }
    if (!res.ok) {
      const text = await res.text();
      throw new Error(`HTTP ${res.status}: ${text.slice(0, 200)}`);
    }
    return await res.json();
  } catch (err) {
    console.error('запрос не удался', err);
    throw err;
  }
}
```

Повторять имеет смысл только безопасные запросы (GET) и ошибки сервера. Повтор POST без ключа идемпотентности может создать дубликат.

Отмена и таймаут

```
const controller = new AbortController();
const timer = setTimeout(() => controller.abort(), 5000);

try {
  const res = await fetch(url, { signal: controller.signal });
  ...
} catch (err) {
  if (err.name === 'AbortError') {
    // запрос отменён: таймаут или новый ввод
    пользователя
  }
} finally {
  clearTimeout(timer);
}

// Короткая запись таймаута
fetch(url, { signal: AbortSignal.timeout(5000) });
```

Отмена обязательна для поиска по мере ввода: иначе ответ на старый запрос может прийти позже нового и перезаписать результат.

Параллельные запросы

`Promise.all([a, b])`

Ждать все; любая ошибка отклоняет результат целиком

`Promise.allSettled([a, b])`

Ждать все и получить статус каждого — удобно для независимых блоков страницы

`Promise.race([a, b])`

Первый завершившийся

`Promise.any([a, b])`

Первый успешный

CORS и куки

Одинаковое происхождение

Совпадение протокола, домена и порта; иначе запрос считается межсайтовым

Access-Control-Allow-Origin

Заголовок сервера, разрешающий обращение с другого адреса

Предварительный запрос OPTIONS

Браузер шлёт его перед «непростыми» запросами: свои заголовки, методы PUT и DELETE

credentials: 'include'

Отправлять куки на другой домен; сервер должен разрешить учётные данные

SameSite

Атрибут куки: Lax по умолчанию, для межсайтовых нужен None и Secure

CORS ограничивает браузер, а не сервер: ошибка возникает на стороне клиента, а чинится настройкой заголовков на сервере.

Старый XMLHttpRequest

```
const xhr = new XMLHttpRequest();
xhr.open('GET', '/api/users');
xhr.responseType = 'json';
xhr.onload = () => {
  if (xhr.status < 400) console.log(xhr.response);
};
xhr.onerror = () => console.error('сетевая ошибка');
xhr.upload.onprogress = e => {
  console.log(Math.round(e.loaded / e.total * 100) + '%');
};
xhr.send();
```

Единственная заметная причина использовать его сегодня — ход отправки файла: у fetch нет простого события прогресса загрузки.

Живые данные без опроса

EventSource (SSE)

Односторонний поток событий от сервера; сам восстанавливает соединение

WebSocket

Двусторонний обмен сообщениями в реальном времени

Периодический опрос

Простой вариант; не забывайте останавливать таймер при уходе со страницы

navigator.sendBeacon(url, data)

Отправка небольших данных при закрытии вкладки

Приёмы для интерфейса

- Показывайте состояние загрузки и блокируйте кнопку — иначе пользователь нажмёт дважды.
- Задержка перед отправкой при вводе (300–500 мс) экономит запросы при поиске.
- Сообщения об ошибках пишите человеку, а подробности — в консоль и журнал.
- Не выводите ответ сервера в разметку через innerHTML без экранирования.
- Токены доступа храните так, как позволяет ваша модель безопасности; куки с флагом HttpOnly недоступны из JavaScript и потому надёжнее localStorage.
- Вкладка «Сеть» в средствах разработчика показывает и предварительные запросы OPTIONS — начинайте отладку CORS с неё.